

QWeave Architecture Mathematical Foundations and Technology Stack

Implementation blueprint with proof obligations and LaTeX-ready mathematics

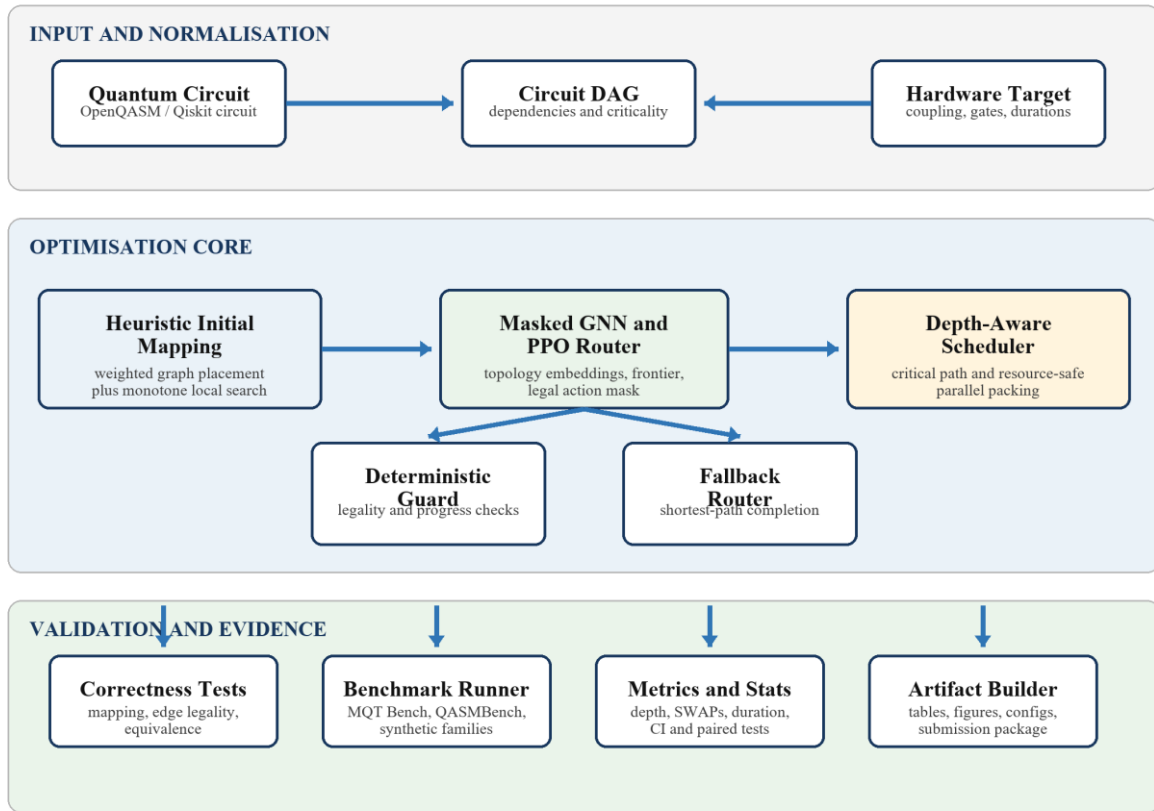
Team: Shrivardhini N | Atharva Sheersh Pandey | Diptesh Das | Haridasu Sreedhar

Guide: Prof. Bhuvaneswari M | Planning horizon: 21 days | Version: 1.0

Executive Design Decision

Build QWeave as a Qiskit-compatible staged compiler with a strict safety boundary around the learned router. The heuristic mapper creates a strong deterministic starting state; the GNN-PPO policy chooses among legal routing actions; a watchdog and shortest-path fallback guarantee progress; the depth-aware scheduler determines realised makespan; and validators reject any circuit that violates connectivity, dependencies, or equivalence.

QWeave Proof-Backed Compiler Architecture



Output: legal scheduled circuit + final layout + evidence bundle + manuscript-ready results

Figure 1. Ideal QWeave architecture. Learned decisions are enclosed by deterministic guards and validation.

1 System Boundaries and Data Contracts

Component	Input	Output	Non-negotiable contract
Circuit normaliser	QuantumCircuit or OpenQASM	Canonical circuit DAG	Preserve classical conditions and declared semantics
Target adapter	Qiskit Target/backend	Canonical hardware graph	Connectivity, direction, gate duration, and error share one snapshot

Component	Input	Output	Non-negotiable contract
Initial mapper	Circuit DAG + hardware graph	Injective mapping and trace	Deterministic under seed; accepted refinements are monotone
Routing environment	Mapping + frontier + look-ahead	Transition and legal-action mask	No illegal action may be sampled or executed
GNN-PPO policy	Graph state and mask	SWAP or EXECUTE action	Policy never bypasses the deterministic guard
Fallback router	Blocked frontier state	Finite legal action sequence	Completes on a connected usable target component
Scheduler	Routed DAG + durations	Start times and ordered circuit	All precedence and resource constraints hold
Validator	Input/output/final layout	Pass/fail evidence	Invalid output is excluded and counted as failure

2 Formal Problem Model

Let the logical circuit induce a dependency DAG and a weighted interaction graph on logical qubits. Let the hardware be a directed coupling graph with instruction properties. At routing step t , the mapping π_t assigns each live logical qubit to a distinct physical qubit.

$$G_C = (L, E_C), \quad G_H = (P, E_H), \quad \pi_t : L \hookrightarrow P$$

The initial mapping minimises weighted hardware distance. Weights combine interaction frequency, temporal proximity, and reverse critical-path rank; all terms must be normalised and fixed before test evaluation.

$$C_{\text{map}}(\pi) = \sum_{(i,j) \in E_C} w_{ij} d_H(\pi(i), \pi(j))$$

The end-to-end training objective is multi-objective, but reporting must keep every metric separate. Depth receives the dominant coefficient; SWAP count, compiler time, and calibration penalty break ties or regularise pathological solutions.

$$J = \alpha D + \beta S + \chi T + \eta E_{\text{cal}}, \quad \alpha \gg \beta, \chi, \eta \geq 0$$

3 Heuristic Initial Mapping

3.1 Construction

1. Build logical weights w_{ij} from two-qubit gate count, a decay by gate distance, and criticality.
2. Rank logical qubits by weighted degree and critical-frontier participation.
3. Rank physical qubits by centrality, usable degree, duration, and optional reliability.
4. Greedily place the highest-pressure logical pair on a short, reliable hardware edge; add remaining qubits by marginal cost.
5. Run bounded pair-swap local search. Accept only strict improvements in the declared mapping cost.

3.2 Proposition 1 Monotone local-search termination

Statement. If each accepted pair swap produces a strict reduction in C_{map} and the number of logical and physical qubits is finite, the refinement process terminates after finitely many accepted moves at a pair-swap local optimum.

$$\Delta C = C_{\text{map}}(\pi') - C_{\text{map}}(\pi) < 0 \implies C_{k+1} < C_k$$

Proof sketch. There are finitely many injective logical-to-physical mappings. The objective sequence is strictly decreasing, so the algorithm cannot revisit a mapping. Therefore only finitely many moves can be accepted. When no candidate pair swap has negative delta C, the current mapping is locally optimal with respect to the chosen neighbourhood. This proves termination and local—not global—optimality.

4 Routing MDP and GNN-PPO Policy

4.1 State

- Physical-node features: resident logical-qubit embedding, next-use distance, criticality, remaining degree, availability time, and optional calibration values.
- Physical-edge features: supported two-qubit instructions, direction, duration, error, occupancy, and distance change for look-ahead interactions.
- Circuit context: embeddings of executable frontier gates and a bounded look-ahead window; global progress and current lower-bound gap.

4.2 Actions and legality

Actions are legal coupling-edge SWAPs and an EXECUTE action when at least one frontier gate is directly supported. The masked softmax assigns exactly zero probability to every action for which $m(s,a)=0$.

$$\tilde{\pi}_{\theta}(a \mid s) = \frac{m(s, a) \exp z_{\theta}(s, a)}{\sum_{a' \in \mathcal{A}} m(s, a') \exp z_{\theta}(s, a')}$$

4.3 Reward and policy optimisation

Use incremental realised-depth and routing penalties, plus potential-based shaping. If the shaping term has the standard $\gamma \Phi(s') - \Phi(s)$ form, the optimal policy of the underlying discounted MDP is preserved under the standard assumptions [R5]. This gives a principled way to add dense progress feedback.

$$r_t = -\Delta D_t - \lambda_S \Delta S_t - \lambda_E \Delta E_t + \gamma \Phi(s_{t+1}) - \Phi(s_t)$$

Train the actor and critic with the clipped PPO surrogate [R3]. Do not claim that PPO converges to a global optimum; treat training stability and performance as empirical questions.

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

5 Depth-Aware Scheduler

The scheduler operates on the complete routed DAG, including inserted routing and direction-correction gates. Compute reverse critical-path rank, maintain resource availability for every physical qubit, and assign the earliest feasible start time to the highest-priority ready operation.

$$s(v) = \max \left\{ \max_{u \prec v} (s(u) + \tau(u)), \max_{q \in Q(v)} A(q) \right\}$$

Two simple lower bounds help interpret results: the routed DAG critical-path duration and the largest accumulated load on any physical qubit. Report the ratio between realised makespan and this lower bound rather than calling the schedule optimal.

$$D^* \geq \max \left\{ \text{CP}(G_C), \max_{q \in P} \sum_{v: q \in Q(v)} \tau(v) \right\}$$

6 Formal Safety and Correctness Results

Lemma 1 Mapping preservation under SWAP

Statement. If π_t is injective and a SWAP exchanges the logical occupants of two distinct physical vertices, π_{t+1} is injective.

Proof sketch. The update is composition with a transposition, which is a bijection on physical positions. Composition of an injection with a bijection remains injective.

Lemma 2 Illegal-action exclusion

Statement. If at least one legal action exists and masked logits are normalised only over actions with $m(s,a)=1$, every illegal action has probability zero.

Proof sketch. The numerator contains the factor $m(s,a)$. It is zero for illegal actions, while the denominator is positive because at least one legal action exists.

Theorem 1 Hardware legality

Statement. Every two-qubit instruction emitted by QWeave is supported by the target on its scheduled physical pair.

Proof sketch. EXECUTE is enabled only when the target guard validates the mapped pair and direction. SWAPs are decomposed using supported instructions. The scheduler does not change operands; therefore all emitted two-qubit operations remain legal.

Theorem 2 Semantic equivalence up to final permutation

Statement. The routed output implements the logical circuit up to the declared initial and final wire permutations.

Proof sketch. Induct over emitted operations. Logical gates are executed on the current images of their operands. Each inserted SWAP updates both the quantum state and the classical mapping by the same transposition, preserving the interpretation of every logical wire. Composition yields the stated permutation relation.

Theorem 3 Scheduler feasibility

Statement. The depth-aware list scheduler preserves every routed-DAG precedence relation and never overlaps two operations using the same physical qubit.

Proof sketch. An operation is selected only from the ready set. Its start time is at least every predecessor completion time and every operand availability time. Updating all operand availability values prevents later conflicts.

Theorem 4 Termination with deterministic fallback

Statement. For a finite circuit on a connected usable coupling graph with enough physical qubits, QWeave terminates when the watchdog invokes a shortest-path fallback after bounded non-progress.

Proof sketch. For the chosen frontier gate, the fallback swaps an operand along a finite shortest path, decreasing its distance to the partner until the gate is adjacent and executes. Thus a frontier gate retires after finitely many fallback steps. The circuit has finitely many gates, so repeated progress terminates.

$$U_{\text{out}} = P_{\pi_T} U_C P_{\pi_0}^\dagger$$

6.1 What the proofs do not establish

- They do not establish globally minimal SWAP count, depth, or duration.
- They do not establish that PPO converges, generalises to every topology, or beats SABRE.
- They do not cover disconnected targets unless the selected component can host the circuit and all routing remains within it.
- They do not establish physical fidelity from an approximate calibration model; that remains an empirical estimate.
- They do not replace executable equivalence and legality tests. The proof assumptions must be enforced by code.

7 Recommended Technology Stack

Layer	Recommended choice	Why	Avoid or defer
Language	Python 3.11	Best compatibility across Qiskit, PyTorch, PyG, Gymnasium, and scientific tooling	A second implementation language during the 21-day build
Compiler framework	Qiskit 2.5.x StagedPassManager, Target, DAGCircuit	Native layout/routing/scheduling concepts and strong SABRE baseline [R8]	Depending on private backend internals
Graph kernels	rustworkx for hot graph paths; NetworkX only for prototyping	Qiskit-aligned performance with a familiar fallback	Premature custom C++
Learning	PyTorch + PyTorch Geometric	Custom message passing over node and edge features [R9]	A fixed MLP that discards topology
Environment	Gymnasium custom Env	Clear reset/step contract, checker, wrappers, vectorisation [R10]	Training logic embedded inside compiler code
PPO	Small in-repo masked PPO implementation	Graph policy and variable edge actions are easier to control and audit	Forcing a library policy API that cannot express edge masks
Exact oracle	OR-Tools CP-SAT for small mapping/scheduling cases	Provides optimum or bounded references for small instances [R11]	Running exact search on the full benchmark
Benchmarks	MQT Bench + QASMBench + synthetic families	Breadth, reproducibility, and controlled topology stress [R6, R7]	Only hand-picked favourable circuits
Validation	Qiskit operators/statevectors + optional MQT QCEC	Final-layout-aware semantic checks	Visual circuit comparison
Data and stats	Parquet/CSV, pandas, SciPy, bootstrap scripts	Transparent paired analysis and regenerable tables	Manual spreadsheet result editing
Experiment config	YAML + dataclasses; frozen config copied into each run	Low ceremony and complete provenance	Over-engineered distributed configuration
Quality	pytest, Hypothesis, Ruff, mypy, GitHub Actions	Fast invariant, property, style, and integration checks	Coverage targets without meaningful assertions
Paper	LaTeX + latexmk; IEEEtran or IOP template after venue freeze	Source-controlled equations, references, and figures	Formatting the final paper only in Word

Version policy. Run a compatibility spike and then pin exact versions in a lockfile. The current IBM documentation uses Qiskit 2.5.1 or newer in its transpiler-stage examples [R8], but the project must pin the exact version that passes its tests. For PyTorch and PyG, follow their compatibility matrix rather than copying an arbitrary pair from a tutorial.

8 Proposed Repository Architecture

```

qweave/
  pyproject.toml          # pinned dependencies and tooling
  configs/                # targets, experiments, training, seeds
  src/qweave/
    ir/                   # circuit DAG and hardware model adapters
    mapping/              # interaction graph and local search
    routing/env.py         # Gymnasium environment
    routing/features.py    # graph-state construction
    routing/policy.py      # GNN actor-critic and masks
    routing/ppo.py         # rollout and clipped updates
    routing/fallback.py    # deterministic shortest-path router
    scheduling/            # critical-path list scheduler
    validation/            # legality, invariants, equivalence
    passes/               # Qiskit pass wrappers and integration
  experiments/            # runners; never hand-edited results
  tests/                  # unit, property, integration, regression
  artifacts/{raw,processed,figures,tables}/
  paper/                  # LaTeX, bibliography, generated figures
  scripts/reproduce_main.sh # one-command principal result rebuild

```

9 Interface Signatures

```

InitialMapper.map(circuit_dag, target, seed) -> MappingResult
RoutingEnv.reset(circuit_dag, target, initial_mapping, seed) -> GraphObservation
RoutingEnv.step(action) -> observation, reward, terminated, truncated, info
Router.route(circuit_dag, target, initial_mapping, policy) -> RoutedCircuit
Scheduler.schedule(routed_dag, target) -> ScheduleResult
Validator.validate(input_circuit, output_circuit, layouts, target) -> Evidence
ExperimentRunner.run(config) -> immutable result.json + circuit artifacts

```

10 Test Architecture

Test class	Examples	When it runs
Unit	mapping update, distances, masks, reward components, critical ranks	Every commit
Property-based	injectivity preserved; illegal actions never selected; schedule intervals disjoint	Every pull request
Metamorphic	renamed qubits or graph isomorphism gives correspondingly renamed result	Nightly or pre-merge
Equivalence	small Clifford/random circuits up to final permutation	Every pull request on bounded set
Regression	fixed circuits and seeds do not worsen beyond declared tolerance	Pre-merge and release
Benchmark	full methods $\times$ circuits $\times$ targets $\times$ seeds matrix	Controlled experiment runs

11 Experiment Matrix and Reporting

Axis	Required levels
Methods	Qiskit SABRE; heuristic + fallback; GNN-PPO + fallback; full QWeave; exact/CP-SAT on small cases
Ablations	No GNN; no PPO; no mapping refinement; no scheduler; no potential shaping
Circuit widths	Small 4-8; medium 9-20; larger 21+ only if runtime permits
Topology families	Path, grid, heavy-hex-like, irregular sparse, and at least one held-out family

Axis	Required levels
Seeds	At least 5; preferably 10 for stochastic methods
Timeout	One identical wall-clock limit per circuit-method pair
Outputs	Per-circuit metrics, aggregate distributions, pairwise deltas, failures, runtime, environment manifest

12 LaTeX Proof Integration

The accompanying `QWeave_Mathematical_Foundations.tex` file contains theorem environments, assumptions, formal definitions, and complete proof sketches. Keep theorem labels stable and reference them from the implementation documentation. When a code change alters an assumption, update both the proof and its linked test in the same pull request.

References and Primary Sources

- [R1] G. Li, Y. Ding, and Y. Xie, 'Tackling the Qubit Mapping Problem for NISQ-Era Quantum Devices,' ASPLOS, 2019. doi:10.1145/3297858.3304023.
- [R2] A. Cowtan et al., 'On the Qubit Routing Problem,' arXiv:1902.08091, 2019.
- [R3] J. Schulman et al., 'Proximal Policy Optimization Algorithms,' arXiv:1707.06347, 2017.
- [R4] J. Gilmer et al., 'Neural Message Passing for Quantum Chemistry,' ICML, 2017.
- [R5] A. Y. Ng, D. Harada, and S. Russell, 'Policy Invariance under Reward Transformations,' ICML, 1999.
- [R6] N. Quetschlich, L. Burgholzer, and R. Wille, 'MQT Bench,' Quantum, vol. 7, 1062, 2023. doi:10.22331/q-2023-07-20-1062.
- [R7] A. Li et al., 'QASMBench: A Low-level QASM Benchmark Suite for NISQ Evaluation and Simulation,' arXiv:2005.13018, 2020.
- [R8] IBM Quantum Documentation, 'Transpiler stages' and 'Transpiler API,' <https://quantum.cloud.ibm.com/docs/en/guides/transpiler-stages> (accessed 4 September 2026).
- [R9] PyTorch Geometric Documentation, 'Creating Message Passing Networks,' https://pytorch-geometric.readthedocs.io/en/latest/tutorial/create_gnn.html (accessed 4 September 2026).
- [R10] Gymnasium Documentation, 'Make your own custom environment,' https://gymnasium.farama.org/main/tutorials/environment_creation/ (accessed 4 September 2026).
- [R11] Google OR-Tools Documentation, 'Constraint Optimization and CP-SAT,' <https://developers.google.com/optimization/cp/> (accessed 4 September 2026).